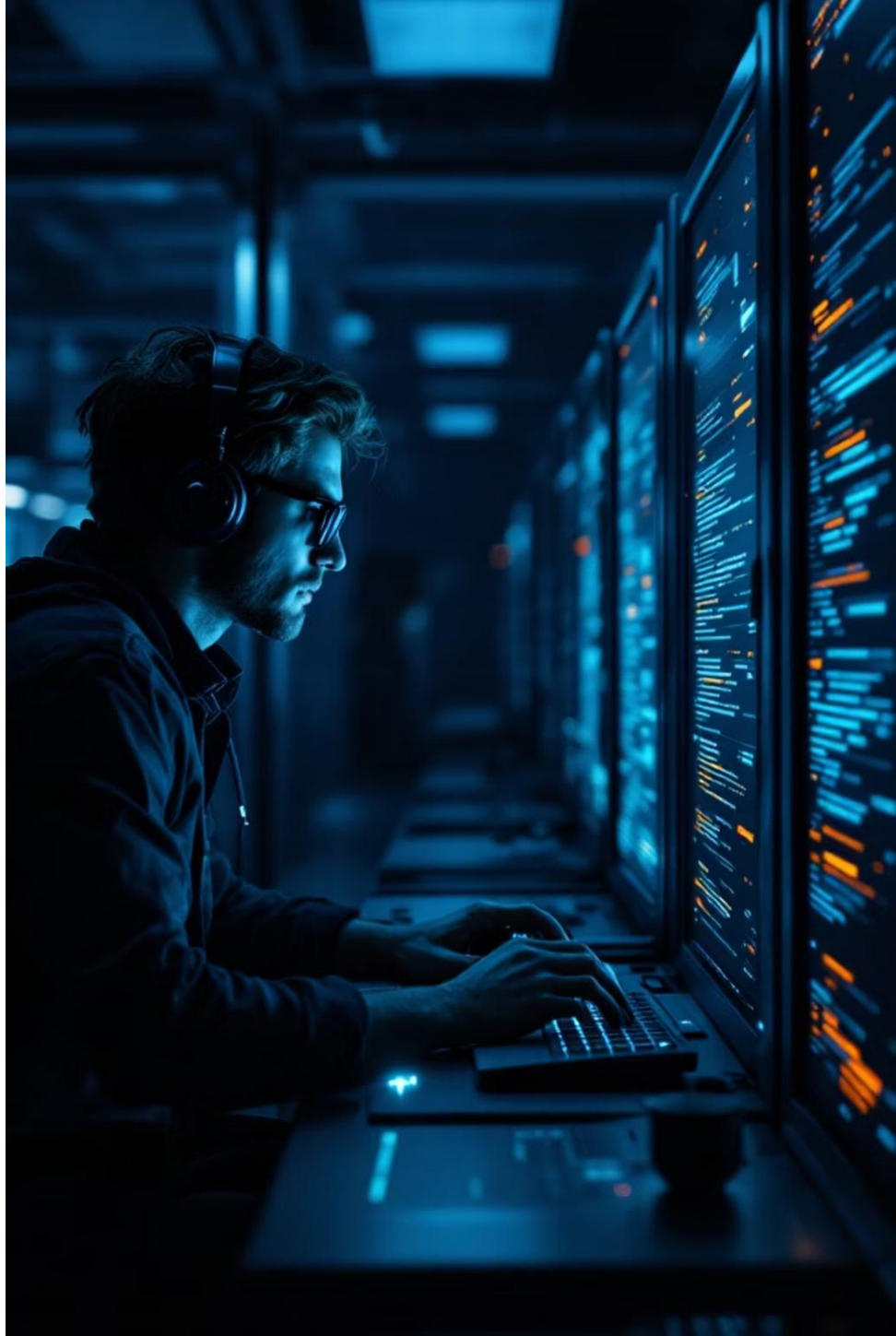
Secure Coding & Defensive Design

Secure coding practices help developers build applications that protect systems from vulnerabilities, malicious input, and security attacks.



Introduction

Secure Coding

Secure coding is the practice of writing software that is resistant to vulnerabilities and attacks. As applications grow in complexity and handle sensitive data, developers must ensure that the code they write protects systems from malicious input, unauthorized access, and unexpected

Defensive Design

Defensive design is a software development approach that assumes users or attackers may attempt to misuse the system. Developers implement safeguards to prevent misuse and minimise damage if vulnerabilities are exploited.

Introduction (Continued)

Both Java-based systems and .NET applications are widely used in enterprise environments and are common targets for attacks such as injection attacks, denial-of-service attacks, and information disclosure.

Implementing secure coding practices helps reduce these risks and improves the overall security of applications.

This document discusses secure coding techniques including **input validation**, **secure error handling**, and **proper logging and monitoring**.

Importance of Secure Coding

Applications often interact with external users, databases, and network services. Attackers exploit weaknesses in application code to gain unauthorised access or disrupt system operations.

Common consequences of insecure coding include:

Data Breaches

Unauthorised system access

Service Disruption

Exposure of sensitive information

Financial & Reputational Damage

Long-term impact on trust and operations

Secure coding practices help developers build applications that are resilient to attacks and capable of detecting suspicious activities.

Validating Inputs

Input validation ensures that data received from users or external systems meets the expected format before it is processed.

Applications must treat all external input as **untrusted**, including:

User Form Inputs

URL Parameters

API Request Bodies

File Uploads

Cookies and Headers

Without validation, attackers can inject malicious data into the system.

Injection Attacks

Injection Attacks

Injection attacks occur when attackers insert malicious input into application queries or commands.

Common types include:

- → SQL Injection
- → Command Injection
- → LDAP Injection
- → XPath Injection

Example of insecure database query:

```
SELECT * FROM users
WHERE username = '" + username + "'
AND password = '" + password + "'";
```

Injection Example

If a malicious user enters:

Malicious Input

username: admin

password: ' OR '1'='1

The query may bypass authentication and return all records.

Prevention Techniques

To prevent injection attacks, developers should use:

- Parameterised queries
- Prepared statements
- Input sanitisation
- Whitelisting allowed input formats

Prepared Statement Example

Example in Java using prepared statements:

```
PreparedStatement stmt = connection.prepareStatement(
    "SELECT * FROM users WHERE username=? AND password=?"
);
stmt.setString(1, username);
stmt.setString(2, password);
```

This prevents attackers from altering the SQL query structure.

Input Validation Techniques

Several techniques can be used to validate input:

1

Whitelisting

Accept only known safe input values.

Example: Accept only numeric IDs. UserID must match: **[0-9]+**

Input Validation Techniques

1

Length Validation

Limit the size of input to prevent buffer overflow or memory issues.

Example: Username length: **5–30 characters**

2

Format Validation

Ensure input follows the correct structure.

Example: Email must match: **example@domain.com**

3

Data Type Validation

Ensure inputs match the expected data type.

Example: Age must be an integer

Secure API Development: Resource Exhaustion, Error Handling & Logging

A technical guide covering how to prevent resource exhaustion attacks, implement secure error handling, and establish safe logging and monitoring practices in modern applications.

Preventing Resource

Exhaustion Attacks

Resource exhaustion attacks occur when attackers send excessive data or requests that consume system resources.

Examples Include

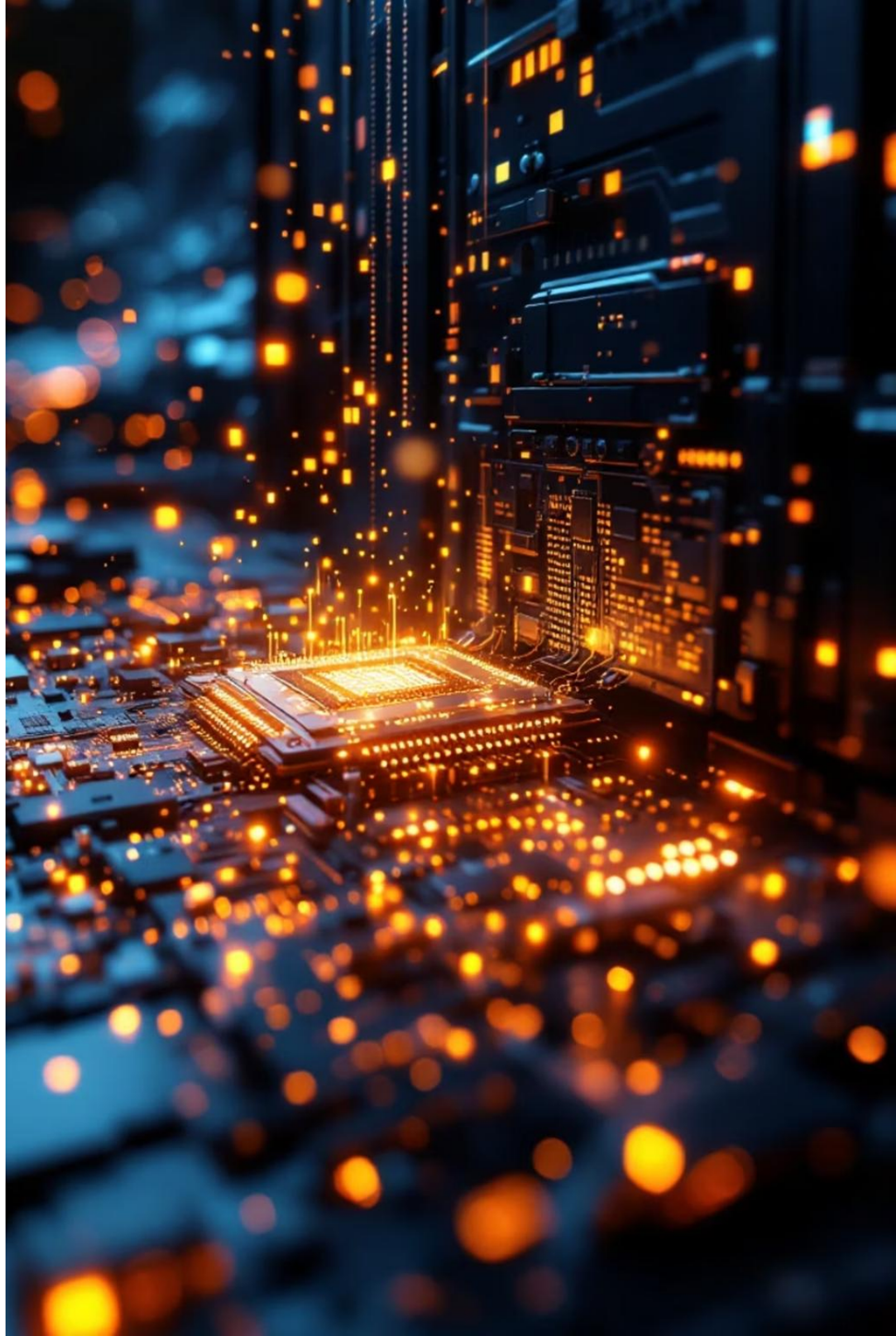
- Large file uploads
- Repeated API calls
- Extremely large input fields

Mitigation Strategies Include

- Request size limits
- Rate limiting
- Timeouts
- Input length restrictions

Example: Limit API request size to: **10 MB maximum request payload**

This prevents attackers from overloading the system.



What is Secure Error Handling?

Error handling refers to how applications respond to unexpected conditions or failures.

If error messages reveal internal system details, attackers may use that information to exploit vulnerabilities.

For example, stack traces may reveal:

- File paths
- Database queries
- Framework versions
- Internal system structure

Why This Matters

This information can help attackers identify weaknesses.

Stack Trace Exposure

Example of insecure error response:

```
java.sql.SQLException: Table 'users' does not exist
at com.example.app.UserService.getUser(UserService.java:52)
```

This reveals:

Database Structure

Table names and schema details are exposed to the attacker.

Application Class Names

Internal class and method names reveal the application architecture.

File Locations

Source file paths and line numbers are disclosed.

Such information should **not** be visible to end users.

Secure Error Handling Techniques

1 Display Generic Error Messages

Users should receive simple error messages.

Example: An unexpected error occurred. Please try again later.

Detailed error information should only appear in internal logs.

2 Centralised Exception Handling

Frameworks such as Java Spring Boot or .NET provide centralised error handling mechanisms.

Example approach:

- Catch exceptions globally
- Log detailed error internally
- Return safe error response to user

3 Avoid Logging Sensitive Information

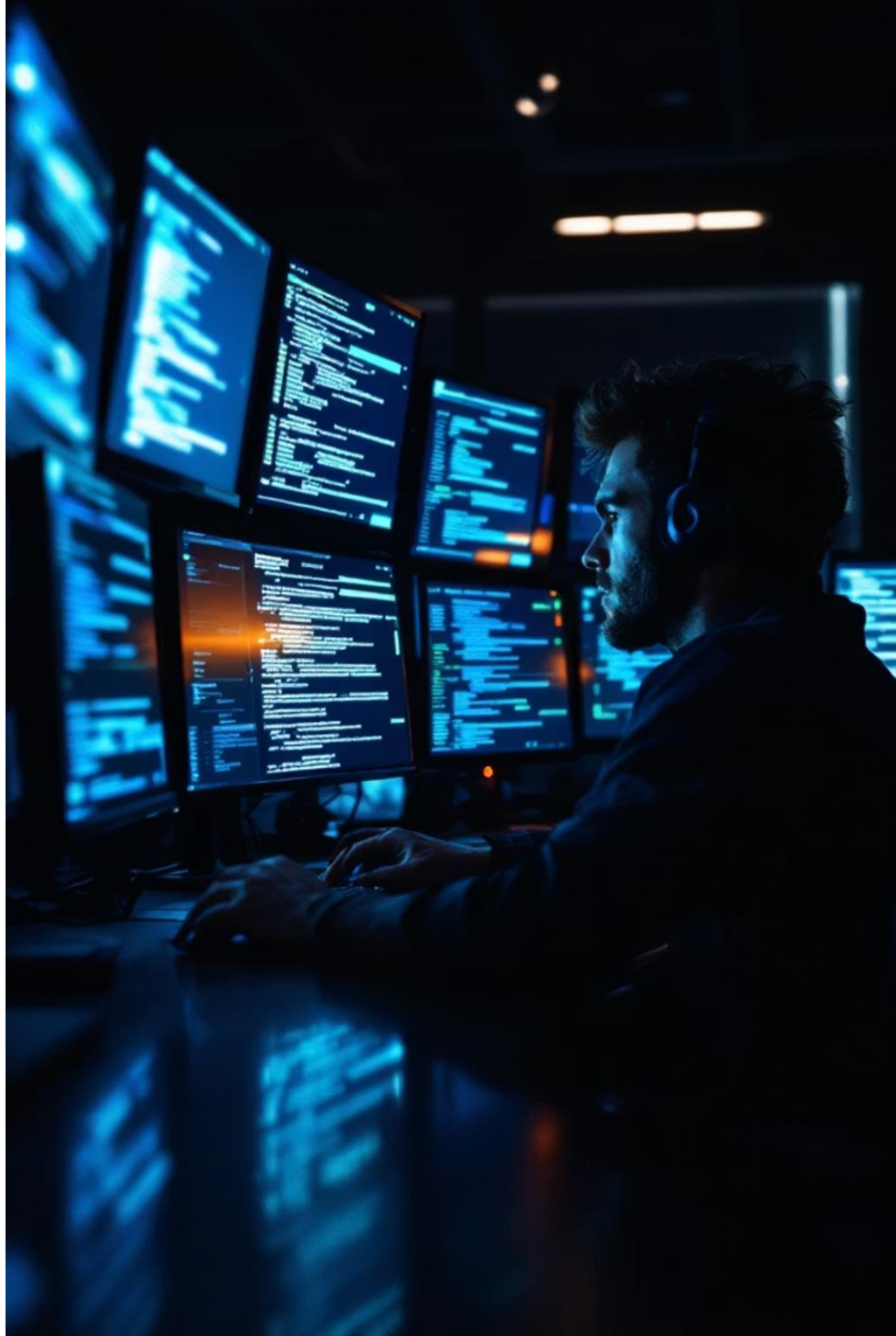
Logs should not include:

- Passwords
- Credit card numbers
- Authentication tokens
- Personal information

Logging and Monitoring

Logging is essential for detecting security incidents and understanding system behaviour.

However, improper logging may expose sensitive information or create security risks.



Purpose of Logging

Logging helps organisations:

Detect Security Attacks

Identify and respond to malicious activity in real time.

Investigate Incidents

Reconstruct events after a security breach or failure.

Monitor System Performance

Track application health and identify bottlenecks.

Maintain Compliance

Maintain compliance with regulations.

What Should Be Logged

Applications should log security-relevant events such as:

Security Events to Log

- User login attempts
- Failed authentication attempts
- Password reset requests
- API access events
- Administrative actions
- System errors

Example Log Entry

Timestamp	10:45 AM
UserID	1052
Action	Failed Login Attempt
IP Address	192.168.1.20

These logs help security teams identify suspicious activity.

What Should NOT Be Logged

Certain information should never be stored in logs.

Sensitive data includes:

- 🔑 Passwords
- 🔐 Private Encryption Keys
- 💳 Credit Card Numbers
- 🎟️ Session Tokens
- 🪪 Full Personal Identification Numbers

Example of insecure log entry:

```
User login attempt: username: admin password: admin123
```

🗒️ This exposes sensitive credentials.

Secure Log Storage

Logs should be protected to prevent tampering or unauthorised access.

Best practices include:

Restrict Log Access to Administrators

Only authorised personnel should be able to view or modify log files.

Encrypt Log Files

Ensure log data is encrypted at rest and in transit.

Store Logs on Separate Servers

Isolate log storage from application servers to reduce risk.

Implement Log Retention Policies

Define how long logs are kept and when they should be securely deleted.

Monitoring and Incident Response

Logs should be continuously monitored to detect abnormal activities.

Security teams may use monitoring tools such as:

SIEM Systems

Security Information and Event Management systems

Intrusion Detection

Intrusion detection systems

Application Monitoring

Application monitoring tools

Example Scenario

If multiple failed login attempts occur within a short time period, the system may trigger an alert for possible **brute-force attacks**.

Example Use Case: Secure Web Application

Consider an online shopping platform built using **Java or .NET**.

Security measures implemented include:

- → *All user inputs are validated before processing.*
- → *Database queries use prepared statements to prevent SQL injection.*
- → *Input length limits prevent large payload attacks.*
- → *Error messages displayed to users are generic.*
- → *Detailed error information is stored only in secure log files.*
- → *Logs record login attempts and administrative actions.*
- → *Sensitive data such as passwords and payment details are never logged.*

> **Result:** The application becomes more resilient to attacks and provides better visibility into security events.

Best Practices for Secure Coding

Developers should follow these best practices:

1. **Untrusted Input**
   Treat all external input as untrusted
2. **Parameterised Queries**
   Use parameterized queries and prepared statements
3. **Input Validation**
   Implement strong input validation
4. **Error Messages**
   Avoid exposing internal system details in error messages
5. **Exception Handling**
   Use centralized exception handling
6. **Security Logging**
   Log security events but avoid logging sensitive data
7. **Log Monitoring**
   Monitor logs regularly for suspicious activity
8. **Security Testing**
   Conduct regular security testing and code reviews

Conclusion

Secure coding and defensive design are critical practices for building safe and reliable applications.

Key Practices

- Validating inputs
- Preventing injection attacks
- Implementing secure error handling
- Maintaining effective logging and monitoring systems

The Outcome

By applying these practices, developers can **significantly reduce the risk of security vulnerabilities**.

Both **Java and .NET** applications benefit from adopting these practices, ensuring that systems remain resilient against attacks and capable of protecting sensitive information.